

The Complete 12-Month DSA Roadmap for Top-Tier SWE Interviews (Google-Level)

Starting point: Absolute beginner. **Target:** Independently solve unfamiliar medium/hard interview problems within 12 months.

How This Roadmap Works

Every phase follows the same 4-step learning loop — do NOT skip steps:

Learn — Understand the data structure/algorithm conceptually. Draw it. Know *why* it works, not just *what* it does.

Implement from scratch — Code the core structure/algorithm yourself, without copying, at least twice, on different days.

Drill patterns — Solve 15–30 curated problems per topic, starting easy, moving to medium, touching a few hard.

Explain out loud — After every problem, explain your approach as if in an interview (record yourself once a week).

This is the step people skip and it's the one that actually builds interview skill.

Weekly structure (roughly 10–14 hrs/week):

3–4 hrs: learn new concept + implement from scratch

5–7 hrs: solve problems (timed, after week 1 of each topic)

1–2 hrs: review/re-solve problems you struggled with 3–7 days later (spaced repetition)

1 hr: explain-out-loud practice or write a short summary note

Problem count target: ~500–600 problems total over the year, but *quality of understanding beats quantity*. Every problem you solve, write 3 lines afterward: (1) what pattern it was, (2) what your first instinct got wrong, (3) the key insight.

Tracking: Keep a spreadsheet with columns: Topic | Problem | Date | Time Taken | Solved Alone? | Pattern | Redo Date.

Redo anything that took >25 min or you couldn't solve, after 1 week and again after 1 month.

PHASE 0 — Foundations (Weeks 1–2)

Goal: build the mental toolkit everything else depends on.

Topics

Big-O, Big-Θ, Big-Ω notation — what it measures, worst/average/best case, why constants and lower-order terms drop out

Time complexity analysis — analyzing loops, nested loops, recursive calls (recurrence relations intuition), amortized analysis (why `ArrayList.add` is $O(1)$ amortized)

Space complexity — auxiliary space vs input space, recursion stack space

Master theorem (intuitive version) — for later use in divide & conquer / recursion analysis

Basic math for DSA — logarithms (why binary search is $O(\log n)$), permutations/combinations basics, modular arithmetic basics (you'll need this for hashing and some math problems), floor/ceiling division

Language proficiency — pick ONE language (Python or Java/C++ are most common for Google) and get fluent in: arrays, string manipulation, built-in collections (list/dict/set or ArrayList/HashMap/HashSet), basic OOP if using Java/C++

Deliverable

Can look at any nested loop and state its time complexity immediately

Can explain amortized analysis in your own words

Comfortable writing clean code in your chosen language without looking up syntax constantly

PHASE 1 — Arrays, Strings & Hashing (Weeks 3-7)

This phase alone is worth spending real time on — it's ~30% of all interview questions and every later pattern builds on it.

1A. Arrays & Strings

Static vs dynamic arrays, how dynamic arrays grow (resizing, amortized $O(1)$ append)

In-place manipulation techniques (reversing, rotating, partitioning)

Prefix sums / difference arrays (huge pattern — range sum queries, range update problems)

Kadane's algorithm (max subarray) — derive it, don't memorize it

String basics: immutability (Java/Python) vs mutability, StringBuilder/list-join tricks, ASCII vs Unicode basics

Two-dimensional arrays / matrices: traversal patterns, in-place rotation, spiral traversal, transpose

1B. Two Pointers

Opposite-direction two pointers (sorted array pair sum, container with most water, trapping rain water)

Same-direction (fast/slow) two pointers (remove duplicates in place, partitioning)

When to recognize it: sorted input, or you need to compare elements from both ends, or in-place partitioning

1C. Sliding Window

Fixed-size window

Variable-size window (expand/shrink based on a condition) — longest substring without repeating characters, minimum window substring, longest substring with at most K distinct characters

When to recognize it: "contiguous subarray/substring" + some min/max/count condition

1D. Hashing

How hash tables work internally: hash functions, collision resolution (chaining vs open addressing), load factor, resizing

Amortized $O(1)$ average-case lookup/insert — and WHY worst case can degrade to $O(n)$

HashMap/HashSet as a pattern-completion tool: frequency counting, complement lookups (two sum), grouping (anagrams), deduplication

Rolling hash (intro here, used later in string matching)

Know how to use built-in sort, and that it's $O(n \log n)$

Understand *why* sorting first often simplifies a problem (turns $O(n^2)$ into $O(n \log n)$)

Problem set focus (~90-110 problems across 1A-1E)

Two Sum family, Best Time to Buy/Sell Stock variants, Product of Array Except Self, Container With Most Water, Trapping Rain Water, 3Sum/4Sum, Longest Substring Without Repeating Characters, Minimum Window Substring, Group Anagrams, Valid Anagram, Longest Consecutive Sequence, Subarray Sum Equals K, Rotate Array/Image, Spiral Matrix, Set Matrix Zeroes, Merge Intervals (preview), Valid Palindrome variants.

Deliverable

Given any array/string problem, you can within 2 minutes identify: is this two-pointer, sliding window, prefix sum, or hashing — and justify why.

PHASE 2 — Recursion Foundations (Weeks 8-9)

Recursion underlies trees, backtracking, DP, and divide & conquer — invest real time here before moving on.

Topics

Base case / recursive case, call stack visualization (draw the call tree by hand for at least 10 problems)

Recursion vs iteration tradeoffs (stack space, readability)

Divide and conquer paradigm: split, solve, combine (merge sort, quick sort, binary search are the canonical examples)

Writing a recurrence relation and reasoning about it informally (you'll formalize this more in DP)

Tail recursion (concept only — most languages in interviews don't optimize it, but know what it is)

Memoization introduction (top-down caching) — the bridge into DP

Problem set focus (~25-30 problems)

Factorial/Fibonacci (for mechanics only), $\text{Power}(x, n)$ via divide & conquer, reverse a string/linked list recursively, subsets via recursion (preview of backtracking), merge two sorted lists recursively, generate parentheses (light preview).

Deliverable

Can write any recursive function by clearly identifying base case + recursive case + what work happens before/after the recursive call

Can hand-trace a recursion tree and count total calls

3A. Sorting Algorithms (know how to implement AND analyze all of these)

Bubble/insertion/selection sort — $O(n^2)$ family, know when insertion sort is actually useful (nearly-sorted data, small n)

Merge sort — full implementation, stability, $O(n \log n)$ guaranteed, $O(n)$ space

Quick sort — partition schemes (Lomuto vs Hoare), average $O(n \log n)$ vs worst $O(n^2)$, why pivot choice matters, in-place property

Heap sort (after you learn heaps in Phase 5 — come back to this)

Counting sort, radix sort, bucket sort — non-comparison sorts, when they beat $O(n \log n)$

Stability of sorting algorithms and why it matters

Know the comparison-sort lower bound — why you can't beat $O(n \log n)$ with comparisons alone (this is a common "why" interview question)

3B. Binary Search — go far beyond "find element in sorted array"

Classic binary search, careful boundary handling (`left <= right` vs `left < right`, off-by-one errors — drill until automatic)

Binary search on answer / "binary search the solution space" — this is the pattern that trips people up most.

Recognize: "minimize the maximum" or "maximize the minimum" type problems (Koko Eating Bananas, Split Array Largest Sum, Capacity to Ship Packages)

Binary search on rotated sorted arrays

Finding first/last occurrence, `lower_bound/upper_bound` style search

Binary search in 2D matrices

Problem set focus (~50-60 problems)

Implement each sort from scratch. Binary Search, Search in Rotated Sorted Array (I & II), Find Minimum in Rotated Sorted Array, Koko Eating Bananas, Split Array Largest Sum, Median of Two Sorted Arrays (hard — attempt after you're comfortable), Search a 2D Matrix, Find Peak Element.

Deliverable

Can implement merge sort and quick sort from memory

Can solve a "binary search on answer" problem by correctly framing it as searching a monotonic predicate space

PHASE 4 — Linked Lists (Weeks 13-14)

Topics

Singly vs doubly linked lists, implement both from scratch (node class, insert/delete/traverse)

Dummy head node technique (removes tons of edge-case bugs — always use it)

Fast/slow pointer (Floyd's cycle detection) — cycle detection, finding the middle, finding cycle start (derive the math, don't memorize)

Reversing a linked list (iterative and recursive), reversing in groups/sub-ranges

Merging linked lists, merge k linked lists (preview — ties into heaps later)

LRU Cache design using a doubly linked list + hash map (classic and very common — build this from scratch)

Reverse Linked List, Linked List Cycle I/II, Middle of Linked List, Merge Two Sorted Lists, Remove Nth Node From End, Reorder List, Copy List with Random Pointer, Add Two Numbers, LRU Cache, Flatten a Multilevel Doubly Linked List.

Deliverable

Can implement a linked list, detect/locate a cycle, and reverse a list (iteratively and recursively) without hesitation
Can build LRU Cache from scratch in under 20 minutes

PHASE 5 — Stacks, Queues & Monotonic Structures (Weeks 15–16)

Topics

Stack: array-based and linked-list-based implementation, LIFO use cases

Queue: array-based (circular buffer), linked-list-based, deque (double-ended queue)

Implement a queue using two stacks and vice versa (classic conceptual question)

Monotonic stack — a critical pattern: next greater/smaller element, daily temperatures, largest rectangle in histogram. Learn to derive *why* a monotonic stack gives $O(n)$ instead of $O(n^2)$

Monotonic queue/deque — sliding window maximum/minimum

Stack-based expression evaluation (valid parentheses, calculator problems, RPN evaluation)

Problem set focus (~30–35 problems)

Valid Parentheses, Min Stack, Evaluate Reverse Polish Notation, Daily Temperatures, Next Greater Element I/II, Largest Rectangle in Histogram, Sliding Window Maximum, Implement Queue using Stacks, Basic Calculator I/II, Asteroid Collision.

Deliverable

Recognize "next greater/smaller element" instantly and implement a monotonic stack solution in $O(n)$

Understand why naive approaches to these problems are $O(n^2)$ and how the monotonic structure eliminates redundant work

PHASE 6 — Trees (Weeks 17–22)

Trees are foundational for a huge fraction of Google-style questions.

6A. Binary Trees

Tree terminology (depth, height, balanced, complete, full, perfect)

Traversals: preorder, inorder, postorder (recursive AND iterative using an explicit stack — do both), level-order (BFS with a queue)

Recursive tree thinking: "what does this function return for a subtree, and how do I combine left/right results" — this is THE core skill for tree problems

Common patterns: height/depth, diameter, balanced check, path sum problems, lowest common ancestor, tree serialization/deserialization, invert/mirror

6B. Binary Search Trees (BST)

BST property, search/insert/delete implementation

Validate BST (and the classic bug of only checking immediate children instead of valid range)

In-order traversal gives sorted order — exploit this

Kth smallest/largest in BST

BST to sorted list / balanced BST from sorted list (bridges into balanced trees)

6C. Balanced Trees & Self-Balancing (conceptual depth, implementation optional but understand rotations)

Why unbalanced BSTs degrade to $O(n)$ — worst case scenario

AVL trees: balance factor, rotations (LL, RR, LR, RL) — understand conceptually, know when Google might ask "how would you keep this balanced"

Red-Black trees — know the properties and that they back many language libraries (Java TreeMap, C++ std::map) — implementation NOT required, but be able to discuss tradeoffs

6D. N-ary Trees & Tries

General tree traversal (N children instead of 2)

Trie (prefix tree) — implement from scratch: insert, search, startsWith. Recognize when a problem screams "trie": prefix matching, autocomplete, word search with multiple words, longest common prefix

6E. Advanced Tree Structures (know they exist, implement at least once each — used for range queries)

Segment Tree — build, point update, range query; understand lazy propagation conceptually

Fenwick Tree / Binary Indexed Tree (BIT) — simpler alternative for prefix sum + point update problems

When to reach for these: "range sum/min/max query with updates" problems

Problem set focus (~90-110 problems)

Max Depth, Diameter of Binary Tree, Balanced Binary Tree, Path Sum I/II/III, Binary Tree Level Order Traversal, Zigzag Level Order, Serialize/Deserialize Binary Tree, Lowest Common Ancestor (binary tree and BST versions), Validate BST, Kth Smallest in BST, Construct Tree from Preorder/Inorder, Binary Tree Maximum Path Sum, Implement Trie, Word Search II, Range Sum Query (Mutable) via Segment/Fenwick Tree.

Deliverable

Can solve most tree problems by writing a recursive function and reasoning about return values, without looking at a template

Comfortable implementing BFS and DFS (both recursive and iterative) traversals

Implemented a Trie and a Segment Tree or Fenwick Tree at least once from scratch

PHASE 7 — Heaps / Priority Queues (Weeks 23-24)

Topics

Binary heap structure (array representation of a complete binary tree), min-heap vs max-heap

Implement a heap from scratch: heapify-up, heapify-down, build-heap in $O(n)$ (know why build-heap is $O(n)$ not $O(n \log n)$ — a common "why" follow-up)

Using built-in priority queue/heap libraries fluently

Top-K pattern — maintain a heap of size K (k th largest, top K frequent elements, K closest points)

Two-heap pattern — median of a data stream (max-heap for lower half, min-heap for upper half)

K-way merge pattern — merge K sorted lists/arrays using a heap

Problem set focus (~25-30 problems)

K th Largest Element in an Array/Stream, Top K Frequent Elements, K Closest Points to Origin, Find Median from Data Stream, Merge K Sorted Lists, Task Scheduler, Meeting Rooms II (heap variant).

Deliverable

Can implement a min-heap from scratch (heapify, insert, extract-min) and explain why operations are $O(\log n)$

Instantly recognize "kth largest/smallest," "top K ," "merge K sorted," "running median" as heap problems

PHASE 8 — Graphs (Weeks 25-31)

One of the richest and most-tested areas for Google specifically.

8A. Graph Representations & Traversal Basics

Adjacency list vs adjacency matrix — tradeoffs (space, edge lookup speed)

Directed vs undirected, weighted vs unweighted, cyclic vs acyclic

BFS: implementation, shortest path in unweighted graphs, level-by-level processing

DFS: recursive and iterative (explicit stack), preorder/postorder concepts applied to graphs

Connected components (undirected graphs)

8B. Topological Sort

DFS-based topological sort (postorder reversal) — derive why it works

Kahn's algorithm (BFS-based, in-degree tracking)

Cycle detection in directed graphs (using DFS with recursion-stack tracking / coloring: white-gray-black)

When to recognize it: dependency ordering, "course schedule," build systems, task ordering

8C. Union-Find (Disjoint Set Union)

Implement from scratch: find, union, with path compression AND union by rank/size

Analyze near- $O(1)$ amortized complexity (inverse Ackermann function — know the name, don't need the full proof)

Use cases: connected components, cycle detection in undirected graphs, Kruskal's MST, "number of islands" variants, dynamic connectivity problems

8D. Shortest Path Algorithms

Dijkstra's algorithm — implement with a min-heap, understand why it fails with negative weights

Bellman-Ford — handles negative weights, detects negative cycles, $O(VE)$

Floyd-Warshall — all-pairs shortest path, $O(V^3)$, DP formulation

0-1 BFS (deque-based shortcut for graphs with only 0/1 weights) — good to know exists

8E. Minimum Spanning Tree (MST)

Kruskal's algorithm (sort edges + union-find)

Prim's algorithm (heap-based, grows a tree)

When MST is the right framing for a problem (network design, "connect all points minimum cost")

8F. Advanced Graph Patterns

Bipartite graph checking (2-coloring via BFS/DFS)

Multi-source BFS (rotting oranges style — start BFS from multiple sources simultaneously)

Grid-as-graph problems (number of islands, flood fill, walls and gates) — recognizing that a 2D grid is just a graph

Union-Find vs BFS/DFS for connectivity — know when each is more natural

Strongly Connected Components (Tarjan's or Kosaraju's) — know conceptually, implement Kosaraju's at least once

Articulation points & bridges (Tarjan's low-link technique) — conceptual understanding, this is edge-of-scope for most interviews but occasionally appears at Google

Problem set focus (~110–130 problems)

Number of Islands, Clone Graph, Course Schedule I/II, Pacific Atlantic Water Flow, Rotting Oranges, Word Ladder, Network Delay Time (Dijkstra), Cheapest Flights Within K Stops, Redundant Connection (Union-Find), Number of Connected Components, Graph Valid Tree, Alien Dictionary (topological sort), Reconstruct Itinerary, Min Cost to Connect All Points (MST).

Deliverable

Can implement BFS, DFS, topological sort (both methods), Union-Find, Dijkstra's, and at least one MST algorithm from scratch without references

Given a novel problem, can identify within a few minutes whether it's a graph problem in disguise (grids, dependency lists, "connections" all count)

PHASE 9 — Backtracking (Weeks 32–34)

Topics

Backtracking = DFS + explicit choice/undo (make a choice, recurse, un-make the choice)

The general template: choices, constraints, goal — and how to reason about the state space

Pruning — why it's the difference between exponential blowup and a workable solution

Subsets/combinations/permutations generation (with and without duplicates — handling duplicates via sorting + skip logic is a critical sub-skill)

Classic constraint-satisfaction problems: N-Queens, Sudoku Solver

Combination Sum family (with/without reuse of elements)

Word Search (DFS + backtracking on a grid, with visited-marking and unmarking)

Partitioning problems (palindrome partitioning)

Problem set focus (~35–40 problems)

Subsets I/II, Permutations I/II, Combination Sum I/II/III, Word Search, N-Queens, Sudoku Solver, Palindrome Partitioning, Generate Parentheses, Letter Combinations of a Phone Number, Restore IP Addresses.

Deliverable

Can write the backtracking template from memory and adapt it to a new problem in minutes

Can correctly reason about time complexity of a backtracking solution (usually exponential, expressed as $\text{branching-factor}^{\text{depth}}$) and explain why pruning helps in practice

PHASE 10 — Greedy Algorithms (Weeks 35–36)

Topics

What makes a problem “greedy-solvable” — the greedy-choice property and optimal substructure; understand that greedy doesn’t always work and *why* (contrast with DP)

Proving greedy correctness informally (exchange argument intuition) — you don’t need formal proofs in interviews, but you must be able to argue convincingly why your greedy choice is safe

Interval scheduling / merge intervals patterns (a huge sub-pattern on its own)

Activity selection, jump game family, gas station problem, task scheduling

Greedy on graphs preview (this connects back to Prim’s/Kruskal’s/Dijkstra’s, which are all greedy algorithms — revisit them through this lens)

Problem set focus (~30–35 problems)

Merge Intervals, Insert Interval, Non-overlapping Intervals, Meeting Rooms I/II, Jump Game I/II, Gas Station, Candy, Partition Labels, Task Scheduler (revisit), Minimum Number of Arrows to Burst Balloons.

Deliverable

Given a new problem, can articulate a candidate greedy strategy and then stress-test it with a counterexample before committing to it (this stress-testing habit is what separates strong candidates)

PHASE 11 — Dynamic Programming (Weeks 37–46)

The single highest-leverage topic for top-tier interviews. Give this the most time of any phase.

11A. DP Foundations

Optimal substructure and overlapping subproblems — identify both explicitly for every problem you solve

Top-down (memoization) vs bottom-up (tabulation) — practice converting between them fluently

How to define a DP state: what does `dp[i]` (or `dp[i][j]`) *mean*, in a full sentence, before writing any code

How to find the recurrence: what smaller subproblems does the current state depend on
Base cases and iteration order (why order matters in bottom-up)
Space optimization: rolling arrays / reducing 2D to 1D when only the previous row is needed

11B. 1D DP

Climbing Stairs, House Robber (I/II), Decode Ways

Longest Increasing Subsequence (LIS) — $O(n^2)$ DP approach AND the $O(n \log n)$ patience-sorting/binary-search approach (derive both)

Maximum Subarray (revisit Kadane's through a formal DP lens)

Word Break (DP + hashing)

11C. 2D DP — Grid & Two-Sequence Problems

Unique Paths, Minimum Path Sum (grid DP)

Longest Common Subsequence (LCS) — the template that unlocks a huge family of two-string problems

Edit Distance (Levenshtein) — derive the recurrence from LCS intuition

Longest Palindromic Subsequence/Substring

Distinct Subsequences

Interleaving String

11D. Knapsack Family

0/1 Knapsack — derive from first principles (include vs exclude each item), then space-optimize

Unbounded Knapsack (coin change, unlimited reuse)

Subset Sum / Partition Equal Subset Sum / Target Sum (all knapsack in disguise — practice recognizing this)

Coin Change I (min coins) and Coin Change II (count combinations) — subtly different DP formulations, know both

11E. Interval DP

Matrix Chain Multiplication (the canonical teaching example — do this even though it rarely appears verbatim, it teaches the pattern)

Burst Balloons, Palindrome Partitioning II (min cuts)

General pattern: $dp[i][j]$ = best answer for subarray/substring from i to j , built from smaller intervals inside

11F. DP on Trees

Diameter of a tree via DP, house robber III (tree version)

General template: compute DP values for children first (postorder), combine at the parent

11G. DP on Grids with States / Bitmask DP

Traveling Salesman-style problems with bitmask state (small n , e.g., ≤ 20)

Recognize "n is small ($\leq \sim 20$) and involves subsets" as a bitmask-DP signal

11H. Digit DP & Probability/Expectation DP (awareness level)

Know these exist as categories; Google interviews touch these rarely, but understand the *shape* of the pattern (state = position + tight constraint for digit DP) so you're not caught completely off guard

Problem set focus (~130–150 problems — spend the most volume here)

Climbing Stairs, House Robber I/II/III, LIS, Word Break, Unique Paths I/II, LCS, Edit Distance, Longest Palindromic Substring/Subsequence, 0/1 Knapsack, Coin Change I/II, Partition Equal Subset Sum, Target Sum, Matrix Chain Multiplication, Burst Balloons, Palindrome Partitioning II, Best Time to Buy/Sell Stock III/IV (state-machine DP), Regular Expression Matching, Wildcard Matching, Interleaving String, Maximal Square, Dungeon Game.

Deliverable

For ANY new DP problem, can explicitly write out: state definition, recurrence, base case, and answer-location — before writing code

Comfortable converting a top-down memoized solution into bottom-up tabulation and vice versa

Can space-optimize an $O(n^2)$ space DP down to $O(n)$ when the recurrence only depends on the previous row/state

PHASE 12 — Bit Manipulation & Math (Weeks 47–48)

Topics

Bitwise operators (AND, OR, XOR, NOT, shifts) and common tricks: check/set/clear/toggle a bit, check power of two, count set bits (Brian Kernighan's algorithm), XOR tricks (finding the single non-duplicate number, swapping without a temp variable)

Bitmask representation of subsets (ties directly back into bitmask DP and backtracking-with-state)

Basic number theory: GCD/LCM (Euclidean algorithm), primality testing, Sieve of Eratosthenes, modular exponentiation

Overflow awareness (especially relevant in Java/C++)

Problem set focus (~20–25 problems)

Single Number I/II/III, Number of 1 Bits, Counting Bits, Power of Two/Three/Four, Sum of Two Integers (bit-level addition), Subsets via bitmask, GCD-based problems.

Deliverable

Can explain and apply the XOR-cancellation trick and Brian Kernighan's bit-counting trick from memory

PHASE 13 — Intervals, Design Problems & String Algorithms (Weeks 49–50)

13A. Intervals (deeper revisit, now combined with everything learned)

Sweep line technique (events sorted by time, processed in order) — used for meeting rooms, skyline problem, employee free time

13B. Design Problems (very common at Google — testing your ability to combine data structures)

LRU Cache (revisit), LFU Cache

Design Twitter / design a rate limiter / design a file system — practice combining hash maps + heaps/lists to satisfy multiple operation constraints

Design a data structure with $O(1)$ insert/delete/getRandom

Iterator design patterns (flatten nested iterator, peeking iterator)

13C. String Algorithms (deeper than Phase 1 hashing)

KMP (Knuth-Morris-Pratt) pattern matching — understand the failure function/partial match table, $O(n+m)$

Rabin-Karp — rolling hash for substring search, and when hashing collisions matter

Manacher's algorithm — $O(n)$ longest palindromic substring (know it exists, expand-around-center is usually sufficient for interviews, but Google occasionally wants the optimal)

Basic string-to-string edit operations (ties back to DP)

Problem set focus (~30–35 problems)

Skyline Problem, Employee Free Time, LFU Cache, Design Twitter, Insert Delete GetRandom $O(1)$, Flatten Nested List Iterator, Implement strStr() (KMP), Repeated DNA Sequences (Rabin-Karp), Shortest Palindrome.

Deliverable

Can design a multi-structure system (e.g., LRU/LFU cache) cleanly, discussing complexity tradeoffs of each internal structure choice

PHASE 14 — Integration, Pattern Mastery & Mock Interviews (Weeks 51–52... but really Months 10–12, see note below)

Important scheduling note: the phases above map to roughly 9–10 months of *learning*. Reserve the **final 2–3 months** primarily for the activities below, overlapping with light review of weak topics — don't cram all mock interviews into the literal last 2 weeks.

14A. The Interview Problem-Solving Framework (memorize and drill this process)

When given an unfamiliar problem, follow this sequence every single time until it's automatic:

Restate the problem in your own words out loud, confirm you understood correctly

Clarify constraints — input size (dictates required complexity), data types, duplicates allowed?, sorted?, negative numbers?, mutable input?, expected output format, edge cases (empty input, single element, all same elements)

Work a small example by hand — write it out, this often reveals the pattern

State a brute-force approach first, out loud, with its time/space complexity — even if obviously suboptimal, this shows process and gives you a correctness baseline

Identify the bottleneck in the brute force — what's the redundant/wasted work? This is the bridge to the optimized approach

Pattern-match: ask yourself — sorted? → binary search/two pointers. Contiguous subarray/substring? → sliding window. Need fast lookup? → hashing. Tree/graph? → BFS/DFS. Optimal value with choices? → DP or greedy. Overlapping subproblems? → DP. K-th / top-K? → heap. Subsets/permutations? → backtracking.

State your optimized approach and complexity BEFORE coding — get verbal buy-in from the interviewer

Code it cleanly — meaningful variable names, modular helper functions, handle edge cases explicitly

Trace through your example by hand against your code to verify

State final time/space complexity, and proactively mention trade-offs or alternative approaches

Invite and handle follow-ups — "what if the input doesn't fit in memory," "what if it's a stream," "what if you need it thread-safe" — practice these explicitly, they're not improvised, top companies ask predictable follow-up categories (scale up input size, streaming data, concurrency, real-world messiness)

14B. Pattern Mastery Review

Go through this checklist and for each, without looking anything up, write the recognition trigger + a code skeleton from memory: Two Pointers · Sliding Window · Fast/Slow Pointers · Merge Intervals · Cyclic Sort · In-place Linked List Reversal · Tree BFS · Tree DFS · Two Heaps · Subsets (Backtracking) · Modified Binary Search · Top-K Elements (Heap) · K-way Merge · Topological Sort · Union-Find · Monotonic Stack · 0/1 Knapsack (DP) · Unbounded Knapsack (DP) · LCS/Edit Distance (DP) · Bitmask · Trie · Sweep Line

14C. Mock Interviews (start around Month 9, ramp up through Month 12)

Months 9–10: 1 mock interview/week (peer, mentor, or a mock-interview platform), focus on process not just correctness

Months 11–12: 2–3 mock interviews/week, mix of topics, include at least a few sessions using Google's actual interview style/environment if possible

After every mock: write down what broke down — was it recognition, implementation speed, communication, edge cases, or complexity analysis? Target that specific gap in your next study block

Practice on a whiteboard or plain-text editor (no autocomplete/syntax highlighting) periodically — Google's interview environment (Google Docs-style) has no IDE support

14D. Company/Role-Specific Prep (last 4–6 weeks)

Review Google's specific interview structure: typically 4–5 rounds of coding/DS&A (some companies mix in a systems design and a googleness/behavioral round) — confirm current format from official sources close to your interview date, since formats evolve

Light system design exposure if your role/level requires it (not the focus of this roadmap, but don't ignore it — check your recruiter's guidance)

Prepare and rehearse behavioral stories (STAR format) — separate track, run in parallel during the last 2 months, not a DSA topic but part of the full interview

Timed full-length practice: simulate a 45-minute session end-to-end, including the framework steps above, not just "solve the problem"

Deliverable (end of 12 months)

You can take a medium/hard LeetCode problem you've never seen, and within 30–45 minutes, work through the entire framework out loud, arrive at an optimal or near-optimal solution, code it cleanly, and correctly analyze its complexity

You have done 15–20+ full mock interviews and can point to concrete, specific improvement over that series

Suggested Month-by-Month Pacing (12 months, adjust to your pace)

Month	Focus
1	Phase 0 (Foundations) + start Phase 1 (Arrays/Strings/Hashing)
2	Finish Phase 1 + Phase 2 (Recursion)
3	Phase 3 (Sorting & Binary Search) + Phase 4 (Linked Lists)
4	Phase 5 (Stacks/Queues/Monotonic) + start Phase 6 (Trees)
5	Finish Phase 6 (Trees, incl. Tries/Segment Trees)
6	Phase 7 (Heaps) + start Phase 8 (Graphs)
7	Continue/finish Phase 8 (Graphs — this is a big phase)
8	Phase 9 (Backtracking) + Phase 10 (Greedy)
9	Phase 11 (Dynamic Programming) — start mock interviews (1/week)
10	Continue Phase 11 (DP) + Phase 12 (Bit Manipulation/Math) — mocks continue
11	Phase 13 (Intervals/Design/Strings) + pattern review — mocks ramp to 2-3/week
12	Full integration, weak-area review, company-specific prep, heavy mock interview volume

Adjust freely — if a phase needs 5 weeks instead of 4, take it; the deliverables matter more than the calendar. Build in 1 buffer week per quarter for catch-up/review.

Complexity Cheat Sheet (keep this visible while studying)

Structure/Algorithm	Time	Space	Notes
Array access / HashMap avg lookup	$O(1)$	—	HashMap worst case $O(n)$
Binary Search	$O(\log n)$	$O(1)$	requires sorted/monotonic
Sorting (comparison-based)	$O(n \log n)$	varies	$O(n^2)$ lower-quality sorts avoid unless small/nearly-sorted
BFS/DFS (graph)	$O(V+E)$	$O(V)$	adjacency list assumed

Structure/Algorithm	Time	Space	Notes
Dijkstra (heap-based)	$O((V+E) \log V)$	$O(V)$	non-negative weights only
Union-Find (with optimizations)	$\sim O(\alpha(n))$ per op	$O(n)$	practically constant
0/1 Knapsack DP	$O(n \cdot W)$	$O(n \cdot W) \rightarrow O(W)$ optimized	
LCS / Edit Distance	$O(n \cdot m)$	$O(n \cdot m) \rightarrow O(\min(n, m))$ optimized	
Backtracking (subsets/permutations)	$O(2^n) / O(n!)$	$O(n)$ stack	pruning reduces practical runtime

Resource Notes (use as supplements, not replacements for this roadmap)

Problem sets: LeetCode (primary), organized by the patterns above rather than random order

For deeper conceptual grounding on any data structure/algorithm you find yourself memorizing instead of understanding, look up a visual explanation (visualgo.net-style animations are excellent for trees/graphs/sorting) before returning to problems

For pattern-based practice, curated pattern lists (e.g., "Grokking the Coding Interview"-style pattern breakdowns, NeetCode's roadmap) align well with the structure above and can serve as a secondary problem source

Keep official/current sources in view for Google's actual interview process and format, since specifics (round count, focus areas) can change — check close to your interview date rather than relying on older anecdotes

Final Notes on the Meta-Skill

The actual goal of this year is not "know 500 problems." It's building a **transferable recognition system**: input shape + constraints $\rightarrow$ candidate patterns $\rightarrow$ brute force $\rightarrow$ bottleneck $\rightarrow$ optimization $\rightarrow$ complexity $\rightarrow$ clean code $\rightarrow$ clear explanation. Every single problem you solve, do it in service of strengthening that pipeline, not just to add a checkmark. If you find yourself pattern-matching from memory instead of re-deriving why the pattern applies, slow down — that's the habit most likely to fail you on a genuinely novel Google problem.

Good luck — this is a serious, well-structured plan. Consistency over 12 months matters far more than any single week of intensity.